

CHOFYAI STUDIO

15 · Riesgos y deuda técnica

Documento	15-risks-and-technical-debt.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/15-risks-and-technical-debt.md
Generado por	scripts/docs/build-pdf.mjs

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

Registro de hallazgos obtenidos leyendo el código. **Este documento es informativo: no se ha corregido nada.** Cada entrada indica evidencia concreta, severidad, impacto, probabilidad y recomendación. Los hallazgos de seguridad se detallan en [11 · Seguridad](#) y aquí sólo se referencian.

Escala usada:

- **Severidad:** crítica (rompe el producto), alta (rompe un flujo importante), media (degrada la experiencia o el mantenimiento), baja (molestia o riesgo latente).
- **Probabilidad:** alta (ocurre en uso normal), media (ocurre en ciertas condiciones), baja (requiere una combinación poco frecuente).

1. Resumen

ChofyAI Studio · 15 · Riesgos y deuda técnica					v0.5.1 (commit f840055) · 2026-08-28				
ID	Hallazgo	Severidad	Probabilidad	Prioridad					
R-01	Cuatro versiones distintas conviviendo en el repositorio	Alta	Alta	1					
R-02	Scripts de Linux referenciados que no existen	Alta	Alta	2					
R-03	Cobertura de pruebas casi nula en los dos archivos principales	Alta	Alta	3					
R-04	Prueba de seguridad que no prueba nada	Alta	Alta	4					
R-05	<code>App.tsx</code> con 2766 líneas y más de veinte componentes	Media	Alta	5					
R-06	<code>kill -9</code> a procesos ajenos en el pre-flight de puertos	Media	Media	6					
R-07	Escritura no atómica de <code>settings.json</code> y <code>processes.json</code>	Media	Media	7					
R-08	<code>settings.json</code> corrupto se descarta en silencio	Media	Media	8					
R-09	Lógica de resolución de rutas duplicada entre Rust y Bash	Media	Media	9					
R-10	Dependencia de utilidades de macOS sin guardas de plataforma	Media	Media	10					
R-11	Campos de manifiesto declarados que el backend ignora	Media	Alta	11					
R-12	Clonado de terceros sin fijar commit	Alta	Media	12					
R-13	Comparación de versiones por orden lexicográfico	Baja	Media	13					
R-14	Duplicación entre <code>start_tool</code> y <code>restart_tool</code>	Baja	Alta	14					

ID	Hallazgo	Severidad	Probabilidad	Prioridad
ChofyAI Studio · 15 · Riesgos y R-15	deuda técnica Errores silenciados de forma sistemática	Media	Alta	v0.5.1 (commit f840055) · 2026-08-28 15
R-16	Un manifiesto inválido deja la lista de herramientas vacía	Media	Baja	16
R-17	Coste de los sondeos periódicos del frontend	Baja	Alta	17
R-18	Contrato de eventos basado en prefijos de texto	Media	Baja	18
R-19	Convivencia de dos árboles de documentación	Baja	Alta	19
R-20	Configuración de ESLint ausente pero referenciada	Baja	Alta	20
R-21	Decisiones pendientes de validación humana	—	—	—

2. Hallazgos detallados

R-01 · Cuatro versiones distintas conviviendo

- **Categoría:** consistencia de release.
- **Evidencia:** `package.json` → `0.5.1`; `src-tauri/Cargo.toml` → `0.5.1`; `src-tauri/tauri.conf.json` → `0.5.0`; `APP_VERSION` en `src/App.tsx` (línea 39) → `0.5.0`.
- **Por qué importa:** `get_system_summary` devuelve `env!("CARGO_PKG_VERSION")`, es decir `0.5.1`, mientras `UpdateChecker` compara el último release publicado contra `APP_VERSION`, es decir `0.5.0`. El usuario puede ver dos versiones distintas en la misma ventana y recibir un aviso de actualización para una versión que ya tiene.
- **Impacto:** alto en confianza; nulo en funcionalidad.
- **Recomendación:** una única fuente de verdad. Leer la versión desde el backend en lugar de la constante, y añadir al CI una comprobación que falle si los cuatro valores divergen.
- **Prioridad:** 1.

R-02 · Scripts de Linux referenciados que no existen

- **Categoría:** promesa de plataforma incumplida.
- **Evidencia:** los cuatro manifiestos multiplataforma declaran `linux-x64: scripts/linux/install-<tool>.sh` con el comentario `# TODO: pendiente`, pero `ls scripts/` sólo devuelve `docs`, `mac` y `win`. Además `platforms:` incluye `linux-x64`, con lo que `platform_supported` devuelve `true` y la herramienta se ofrece como instalable.

- **Por qué importa:** en Linux el usuario ve la herramienta disponible, pulsa *Instalar* y recibe `No existe`

script: La promesa se rompe en el peor momento.

Chofre Studio · 15 · Riesgos y deuda técnica

v0.5.1 (commit f840055) · 2026-08-28

- **Impacto:** alto para cualquier usuario de Linux.
- **Recomendación:** quitar `linux-x64` de `platforms`: hasta que existan los scripts, o añadir en CI una validación de que todo script referenciado exista.
- **Prioridad:** 2.

R-03 · Cobertura de pruebas casi nula donde más se necesita

- **Categoría:** pruebas.
- **Evidencia:** 20 pruebas de frontend, todas sobre `utils.ts` e `i18n.ts` (348 líneas combinadas), y 4 pruebas en Rust. `src/App.tsx` (2766 líneas) y `src-tauri/src/system.rs` (2029 líneas) concentran la lógica y no tienen prácticamente cobertura. Los scripts de shell no tienen ninguna.
- **Impacto:** cualquier refactor de rutas, procesos o cola es una apuesta.
- **Recomendación:** la lista priorizada está en [12 · Pruebas y calidad](#), sección 9. Empezar por las funciones puras de `system.rs`, que son baratas de probar.
- **Prioridad:** 3.

R-04 · Una prueba de seguridad que no prueba nada

- **Categoría:** falsa sensación de cobertura.
- **Evidencia:** `delete_model_rejects_path_traversal` en el módulo de pruebas de `system.rs` se limita a comprobar que la cadena `".."` contenga `".."`. No invoca `delete_tool_model` ni ninguna función de producción.
- **Por qué importa:** el nombre sugiere que la guardia de recorrido de rutas —el control de seguridad más importante del backend— está verificada. No lo está.
- **Recomendación:** extraer la guardia a una función pura y probarla con rutas maliciosas reales sobre un directorio temporal.
- **Prioridad:** 4.

R-05 · `App.tsx` como archivo monolito

- **Categoría:** mantenibilidad.
- **Evidencia:** 2766 líneas con más de veinte componentes, todo el estado de la aplicación en un solo `useState` por concepto y ninguna separación por módulo.
- **Impacto:** revisiones difíciles, conflictos de fusión frecuentes, imposibilidad práctica de probar la interfaz.
- **Recomendación:** extraer por bloques con bajo riesgo —primero los modales y paneles, que ya reciben props explícitas—, y sólo después la lógica de la cola.
- **Prioridad:** 5.

R-06 · `kill -9` a procesos ajenos

- **Categoría:** seguridad y disponibilidad. Detalle en [11 · Seguridad](#) como S-02.

- **Evidencia:** bloque de pre-flight de `start_tool` : recorre la salida de `lsof -ti :PORT` y mata con `-9` todo PID que no esté en el registro.
- **Impacto:** una aplicación del usuario que ocupe uno de los cinco puertos declarados muere sin aviso ni oportunidad de guardar.
- **Recomendación:** pedir confirmación, o limitarse a procesos cuyo comando coincida con lo esperado, o usar `SIGTERM` antes que `SIGKILL` .
- **Prioridad:** 6.

R-07 · Escritura no atómica del estado

- **Categoría:** integridad de datos.
- **Evidencia:** `save_settings_to_disk` y `persist_registry` hacen `fs::write` directo sobre el archivo final.
- **Impacto:** un corte de energía o un cierre forzado durante la escritura deja el archivo truncado.
- **Recomendación:** escribir a un archivo temporal en el mismo directorio y renombrar, que es atómico en el mismo sistema de archivos.
- **Prioridad:** 7.

R-08 · Configuración corrupta descartada en silencio

- **Categoría:** experiencia y diagnóstico.
- **Evidencia:** `load_settings` envuelve la lectura y el parseo en condicionales sin rama de error; ante cualquier fallo devuelve la configuración por defecto completa.
- **Impacto:** el usuario pierde su `studio_home` y sus overrides sin ninguna señal, y el siguiente guardado sobrescribe el archivo dañado, borrando la posibilidad de recuperarlo.
- **Recomendación:** distinguir "no existe" de "no parsea"; en el segundo caso, renombrar el archivo a `.bak` y avisar en la interfaz.
- **Prioridad:** 8.

R-09 · Lógica de rutas duplicada entre Rust y Bash

- **Categoría:** duplicación.
- **Evidencia:** `resolve_effective_home` y `path_is_usable` en `system.rs` tienen su espejo en `resolve_studio_home` y `_path_is_usable` de `scripts/mac/common.sh` , y otro más en `ResolveStudioHome` de `scripts/win/common.ps1` . El propio comentario del código admite que son espejos.
- **Impacto:** tres implementaciones de la misma regla que pueden divergir. La versión Bash, por ejemplo, no intenta montar el sparsebundle.
- **Recomendación:** que Rust pase siempre la ruta ya resuelta por variable de entorno — cosa que ya hace — y que los scripts se limiten a usarla sin volver a decidir. La lógica de reserva en Bash sólo debería aplicarse cuando el script se ejecuta a mano.
- **Prioridad:** 9.

R-10 · Dependencia de utilidades de macOS sin guardas de plataforma

- **Categoría:** portabilidad.

- **Evidencia:** `sysctl`, `vm_stat`, `top`, `df`, `lsof`, `osascript`, `open` y `hdiutil` se invocan desde `system.rs` sin `cfg!(target_os)`, salvo `open_in_system`, que sí está detrás de `#[cfg(target_os = "macos")]`. `list_external_volumes` recorre `/Volumes`, que sólo existe en macOS.
- **Impacto:** en Windows o Linux esos comandos fallan y las funciones devuelven ceros o listas vacías. No hay error, pero la información es falsa.
- **Recomendación:** encapsular las lecturas del sistema tras un rasgo por plataforma, o declarar explícitamente que las estadísticas sólo funcionan en macOS.
- **Prioridad:** 10.

R-11 · Campos de manifiesto declarados que el backend ignora

- **Categoría:** contrato engañoso.
- **Evidencia:** los YAML de `apps/` contienen `python_manager`, `healthcheck`, `install` y `notes`; `RawManifest` no declara ninguno de los cuatro y `serde_yaml` los descarta en silencio.
- **Impacto:** quien escriba un manifiesto nuevo creará que `healthcheck` configura la comprobación de salud —no lo hace: la salud es siempre PID vivo más puerto abierto— o que `python_manager: uv` fuerza el gestor —tampoco: lo decide `detect_uv` en Bash—.
- **Recomendación:** implementarlos o eliminarlos de los YAML y de `../MANIFEST_SPEC.md`. Mientras tanto, están documentados como inertes en [07 · Base de datos y persistencia](#).
- **Prioridad:** 11.

R-12 · Código de terceros clonado sin fijar commit

- **Categoría:** cadena de suministro. Detalle en [11 · Seguridad](#) como S-01.
- **Evidencia:** los siete `git clone` de `scripts/mac/*.sh` no fijan etiqueta ni commit; `update_tool` ejecuta `git pull --ff-only`.
- **Impacto:** lo que se instala hoy y lo que se instala mañana pueden ser cosas distintas; una instalación deja de ser reproducible, que es precisamente lo que el proyecto promete.
- **Recomendación:** añadir un campo de commit o etiqueta al manifiesto y usarlo en el clone.
- **Prioridad:** 12.

R-13 · Comparación de versiones por orden lexicográfico

- **Categoría:** corrección.
- **Evidencia:** en `UpdateChecker`, `if (remote != local && remote > local)` compara cadenas.
- **Impacto:** funciona con `0.5.x`, pero `"0.10.0" > "0.9.0"` es falso, de modo que al llegar a la versión 0.10 dejará de avisar de actualizaciones.
- **Recomendación:** comparar por componentes numéricos.
- **Prioridad:** 13.

R-14 · Duplicación entre `start_tool` y `restart_tool`

- **Categoría:** duplicación.

- **Evidencia:** `start_tool` (línea 1607) y `restart_tool` (línea 1738) repiten la resolución de manifiesto, rutas y comando, la creación del log y el `spawn`. La diferencia real es que `restart_tool` mata primero y no hace pre-flight de puerto ni valida `installed_if`.
- **Impacto:** las mejoras aplicadas a uno no llegan al otro. Hoy mismo, reiniciar no valida la integridad de la instalación mientras que arrancar sí.
- **Recomendación:** extraer un `spawn_tool` común y que ambos comandos lo usen.
- **Prioridad:** 14.

R-15 · Errores silenciados de forma sistemática

- **Categoría:** diagnosticabilidad.
- **Evidencia:** 15 usos de `let _ = ...` y 11 de `unwrap_or_default()` en `system.rs`; tres `catch` vacíos en `App.tsx`; dos `expect("stdout piped")` que provocarían pánico si la tubería no existiera.
- **Impacto:** fallos que no dejan rastro. El caso más visible es el auto-montaje del sparsebundle: si `hdiutil` falla, no se registra nada y el usuario sólo ve que está en la ruta de reserva.
- **Recomendación:** registrar en `crash.log` los fallos silenciados de las rutas importantes, empezando por el montaje y la persistencia del registro de procesos.
- **Prioridad:** 15.

R-16 · Un manifiesto inválido vacía toda la lista

- **Categoría:** robustez.
- **Evidencia:** `collect_manifests` usa `?` sobre el resultado de `serde_yaml::from_str`, de modo que un solo YAML mal formado hace fallar la función completa y `list_tools` devuelve error.
- **Impacto:** el usuario ve cero herramientas por culpa de un archivo, sin saber cuál.
- **Recomendación:** omitir el manifiesto defectuoso, seguir con el resto y reportar el archivo problemático en la interfaz.
- **Prioridad:** 16.

R-17 · Coste de los sondeos periódicos

- **Categoría:** rendimiento.
- **Evidencia:** en `App` conviven cuatro intervalos —estadísticas cada 3 s, salud cada 5 s, herramientas cada 8 s, huérfanos cada 60 s— más un tick de 1 s mientras hay instalaciones. El efecto de salud depende de `[tools, startingTools]`, así que se desmonta y remonta cada vez que cambia `startingTools`. Cada `get_system_stats` lanza cuatro procesos del sistema, y cada ciclo de huérfanos lanza un `lsdf` por puerto declarado.
- **Impacto:** consumo constante en reposo; se nota más en portátiles con batería.
- **Recomendación:** pausar los sondeos cuando la ventana no está enfocada y estabilizar las dependencias del efecto de salud con una referencia mutable.
- **Prioridad:** 17.

R-18 · Contrato de eventos basado en prefijos de texto

ChofyAI Studio · 15 · Riesgos y deuda técnica

v0.5.1 (commit f840055) · 2026-08-28

- **Categoría:** acoplamiento frágil.
- **Evidencia:** el frontend decide si una instalación fue bien con `line.startsWith('OK:')`; el backend construye esa línea con `format!("OK: {} instalado", ...)` o `format!("ERROR: instalacion fallo para {}", ...)`.
- **Impacto:** cambiar el texto en Rust —traducirlo, por ejemplo— rompe la interfaz sin ningún error de compilación ni de prueba.
- **Recomendación:** añadir un campo booleano al payload del evento.
- **Prioridad:** 18.

R-19 · Dos árboles de documentación conviviendo

- **Categoría:** deuda documental.
- **Evidencia:** `docs/` contiene 22 documentos previos, incluido `architecture.md`, y ahora existe además `docs/system-documentation/` con 20 documentos, entre ellos [03 · Arquitectura](#).
- **Impacto:** dos fuentes sobre los mismos temas que envejecerán a ritmos distintos.
- **Recomendación:** decidir cuál es la fuente de verdad por tema y dejar en la otra un enlace en lugar de contenido duplicado. Requiere una decisión del responsable del proyecto.
- **Prioridad:** 19.

R-20 · Configuración de ESLint ausente pero referenciada

- **Categoría:** herramientas.
- **Evidencia:** `src/App.tsx` contiene `// eslint-disable-next-line react-hooks/exhaustive-deps`, pero no hay archivo de configuración de ESLint ni dependencia en `package.json`.
- **Impacto:** el comentario no suprime nada porque no hay linter, y además señala una dependencia de efecto incompleta que nadie está verificando.
- **Recomendación:** añadir ESLint con el plugin de hooks, o retirar el comentario y revisar a mano esa dependencia.
- **Prioridad:** 20.

R-21 · Decisiones que requieren validación humana

No son defectos: son preguntas abiertas cuya respuesta corresponde al responsable del proyecto.

1. **Soporte de Windows y Linux:** ¿se completa, se retira de los manifiestos, o se declara explícitamente como no soportado en la interfaz? Hoy el estado intermedio es lo que genera R-02 y R-10.
2. **Marketplace:** `import_marketplace_tool` genera manifiestos **incompletos a propósito**, sin `install_script` ni `run`, que el usuario debe terminar a mano. ¿Es el comportamiento deseado o un estado transitorio?
3. **Pasos stub de los workflows:** ¿se implementan, se documentan como plantillas o se retiran? Hoy se ejecutan y devuelven una nota.
4. **Procesos hijos que sobreviven al cierre:** es deliberado, pero no se avisa al usuario. ¿Debería preguntarse al cerrar la ventana?

5. `settings.json` versionado con rutas del autor: ¿se sustituye por un archivo de ejemplo?

3. Qué no es deuda

Para no confundir decisiones deliberadas con descuidos, estas elecciones están justificadas en el propio código y no deberían "corregirse" sin entender el motivo:

Decisión	Por qué es correcta aquí
Leer estadísticas con <code>sysctl</code> , <code>vm_stat</code> , <code>top</code> y <code>df</code> en vez de añadir una dependencia	Mantiene el árbol de dependencias de Rust en seis crates; el coste es la portabilidad, ya registrada en R-10
Instalación en Bash en lugar de en Rust	Los scripts se pueden ejecutar y depurar a mano, que es exactamente lo que se necesita cuando una instalación falla
Calcular <code>installed</code> en cada llamada en lugar de guardarlo	La verdad la tiene el sistema de archivos; un caché aquí sería una fuente de inconsistencias
Fallback de <code>uv</code> a <code>pip</code>	Permite funcionar sin <code>uv</code> sin degradar el resultado
Procesos hijos que sobreviven al cierre	Es un producto de escritorio, no un supervisor: matar el trabajo del usuario al cerrar la ventana sería peor
Tolerancia de 60 segundos antes de declarar una herramienta caída	Nace de un problema real: cargar modelos grandes lleva tiempo
Post-validación de <code>installed_if</code> tras un script "exitoso"	Es la mitigación de un incidente documentado; parece redundante y no lo es
Sin base de datos	Todo el estado es inspeccionable con un editor de texto, lo que simplifica enormemente el soporte